Language and dialect recognition for cuneiform texts

Quentin André[*†], Maëlle Gabens[*]

[*]IMT Atlantique, Technopole, Plouzané, quentin.andre@imt-atlantique.net
[†]Lab-STICC, UMR CNRS 6285, Brest F-29238, France

1 Abstract

Our work is based on a challenge launched by VarDial in 2019 named Cuneiform Language Identification (CLI). We have done some bibliographic research on the database used for the competition and the several methods tried by the competing teams. In this paper we detail in particular two methods, focusing on two parts, the language representation (N-grams and word embeddings respectively) and their associated estimators (HeLI and BERT respectively). We then reproduced the HeLI algorithm and explored a new solution, the Gated Recurrent Network (GRU) to answer the competition problem. Finally, we will give some insights about our results, and then some leads that could be explored in the future.

2 Introduction

In the 3rd millennium BC, the cuneiform writing was invented in the Mesopotamian region, which corresponds to the modern Iraq, Iran, Syria and Turkey. It was one of the first writings of the world, and it was used for about 4 millenniums. The cuneiform writing system consisted of signs printed on fresh clay tablets. Clay is a material that can withstand the passage of time without suffering significant deterioration, this is why archaeologists have managed to find large quantities of these tablets in a very good state of preservation for such ancient texts. Initially used to write the Sumerian language, it was later adapted to write several other languages of the region, notably Akkadian or Babylonian. Plus, this languages can be distinguished in several different dialects, because they evolved along time and across the Mesopotamian cities and empires [1].
On this paper we will focus on a challenge launched by VarDial in 2019, the Cuneiform Language Identification task [11], which consists in the identification of seven of these languages and dialects: Sumerian and six dialects of Akkadian language, namely Old Babylonian, Middle Babylonian Peripheral, Standard Babylonian, Neo-Babylonian, Late Babylonian, and Neo-Assyrian. Dialect identification is the basis of a wide range of Natural Language Processing tasks (NLP) and is a real challenge for state-of-the-art machine learning algorithms. Moreover, only a few language identifications have been made on languages having symbol characters for content, and the CLI task is the first one on cuneiform writing [2].
Even more interesting, the cuneiform writing was transformed from a purely logo-graphic language (one symbol for one object), to a logo-syllabic language (a word can be either a symbol or a combination of several symbols, which then become syllables), and since there are no punctuation or spaces, tokenization of words is a whole task, which highly depends on the language. Thus, the methods we will present are character-based and not word-based as it is mostly shared among NLP tasks [2].
We worked on this project within the framework of the research project proposed by IMT Atlantique. Our goal was to do a bibliographic work to know where the state of the art was on the subject. We decided to push the project by testing the programs we had studied and by implementing another solution.

3 Dataset

The dataset used by VarDial for its challenge was built by Jauhaunien et al. from about 1800 clay tablets of the Open Richly Annotated Corpus (ORACC[1]). It is an open data project where one can find cuneiform tablets annotated by re-

[1] http://oracc.museum.upenn.edu/

searchers, giving notably the "transliterated form", i.e. a transcription of what is written in latin language.
The dataset of the challenge only uses this transliterated form taken as input for an algorithm which translates it into Unicode characters, as defined in the Unicode norm by Cohen et al. [3]. Thus, every other annotation written by the researchers are lost in the process, as well as the shape of the cuneiform sign, whose graphy also evolved through time, but is not kept in the unicode encoding. Another choice made was to delete every missing or unreadable sign from the texts.
Finally, every text is divided in monolingual lines (as shown in **Figure 2**), and these lines are distributed between a train, development and test sets, with their language distribution shown on **Figure 1**. One can notice that the training set is the only one to be unbalanced. Another flaw of the training set is the high number of duplicates (about 30%) [11].

Language or dialect	acronym	training	validation	test
Sumerian	SUX	53673	668	985
Old Babylonian	OLB	3803	668	985
Middle Babylonian perypheral	MBP	5508	668	985
Standard Babylonian	STB	17817	668	985
Neo-Babylonina	NEB	9707	668	985
Late Babylonian	LTB	15947	668	985
Neo-Assyrian	NEA	32966	668	985

Figure 1: Distribution of lines for each dialect and each data set

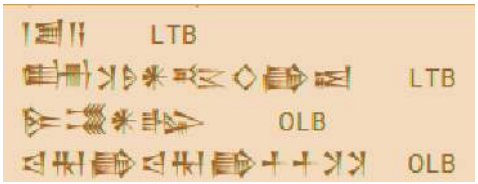


Figure 2: Three lines of the training set and their label

Now, let's see how machine learning was applied on this data.

4 Machine learning for language identification

4.1 Preprocessing and data augmentation

As the quality of data and the useful information that can be derived from it directly affects the ability of our model to learn, the first step is to preprocess the data. In our case, the characteristics of the dataset lead some teams to try several preprocessing methods.
The first one was to delete the duplicates and it was applied by most of the teams [8].
Then, some teams tried to segment the lines into words. However the usual techniques are based on the dialect, and here the preprocessing has to be dialect agnostic, i.e. the algorithms used are unsupervised. These attempts were fruitless, so we will consider only character-based algorithms as for now [8].
Finally, in order to reduce the gap between the number of samples per classes, a technique named adaptation was used: a first classification is made on the training set (using development set for tuning). Then, for every sample of the test set, a confidence score is computed, and all the samples with a confidence score higher than a defined threshold are added to training set with the predicted target. At the end, the remaining test samples are classified a second time (and one can iterate this process) [10].

4.2 Language representation

These preprocessing methods can be used before the language representation, whose goal is to make text usable by other algorithms. Here, every lines of symbols were transformed into matrix of numbers

4.2.1 Ngrams

A n-gram is a subset of n contiguous signs of a line and its frequency in the line. For example, the unigrams of 'happy' would be ['h': 1, 'a': 1, 'p': 2, 'y': 1], and a 2-gram would be ['ha': 1, 'ap': 1, 'pp': 1, 'py': 1].

4.2.2 Embeddings

Another language representation are the embeddings. The principle of word embeddings is to represent each symbol with a vector such that the cosine similarity between the vectors indicates the semantic similarity between the symbols represented by those vectors [9], as represented in the example shown in **Figure 3**.

To achieve this goal, methods like word2vec [9] use neural networks with an unsupervised methods for training like Continuous Bag Of Word (CBOW, **Figure 4**): the output layer predicts which symbol was masked in the input.

4.3 Classifiers

These language representations can be used by the classifiers, i.e. an algorithm which learns how to

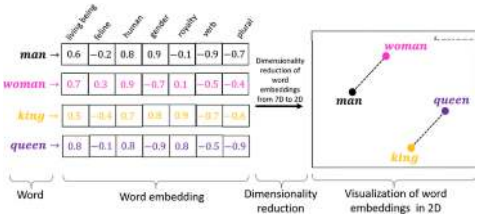


Figure 3: Example of embedding: Take the word "king", replace "man" by "woman" in its semantic, and it becomes "queen". In the embedding space, it corresponds to a simple mathematical formulation: king - man + woman = queen.

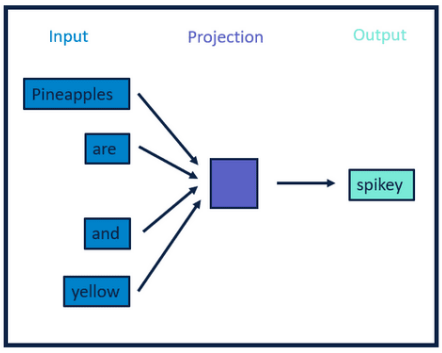


Figure 4: CBOW process example

predict if a sample belongs to a class (here the language). We will present three of them.

4.3.1 HeLI

HeLI [5] is a supervised-learning language identification method using n-grams as language representation. HeLI takes 2 parameters nmax the the max lentgh of the n-grams and p a penalty use for the scores. HeLI scores a line L for a language g using n-grams from 1-grams to nmax-grams. In order to do so, we create a model for each language. For each language and each n in [1,nmax], a score R is computed for each n-gram f following this equation:

$$R(g,f) = -log\frac{c(C_g,f)}{l_{C_g^F}} \quad (1)$$

where c(Cg, f) is the count of the feature f in the training corpus C_g of the language g and $l_{C_g^F}$ is the total number of occurrences of all the n-grams of the same length in the training corpus. As smoothing, in case the count of a feature is zero in some languages, this version of the HeLI method uses the default score p.
Then, to predict a text's language, we use a function $d_g(L,f)$ counting n-grams in line L found for a language g, the final score for each language is the following:

If $d_g(L,f) > 0$:

$$score(L,n) = \frac{1}{d_g(L,f)} \sum_{i=0}^{len(L)-n} R(g,f) \quad (2)$$

else: score(L,n-1)

HeLI finally predicts the language with the lowest score for a line.

4.3.2 BERT

The NRCC team used a slightly modified version of the Bidirectional Encoder Representations from Transformers (BERT) [6] is a deep neural network taking as input lines of symbols and composed of three parts.
The first one realizes the embedding and outputs the encoded signs for the line.
Then, a stack of bidirectionnal Transformer modules encode the input sequence: the context of the word (i.e. the other symbols of the line) are taken into account, and the output are the transformed vectors with the same index.
Finally, an output layer is used: for each phase of training, we use a different one.
For the training, they pre-trained the model on unlabeled text using 2 pre-training tasks: a Masked Language Modelling (MLM, **Figure 5**), i.e. the output layer has to predict the full phrase while one of the word is masked in the input phrase, and a sentence pair classification (SPC): the aim of the SPC is to predict whether 2 lines belong to the same class, and thus the output layer is a binary classifier and takes as input the joint encoding of 2 lines.

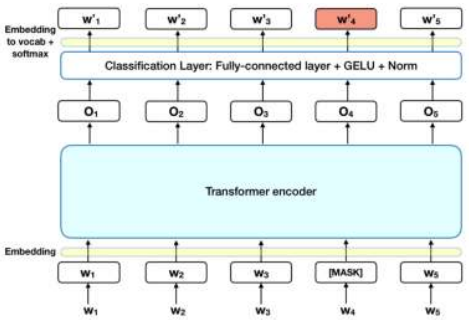


Figure 5: MLM process scheme

The model is pre-trained again with a MLM, this time having for train set every data (train, dev, test).
Finally, the language identification classifier is fine-tuned [4].

4.3.3 GRU

We tried to implement another algorithm than the ones presented in the competition: GRU (Gated Recurrent Unit) [7], a neural network using embeddings.
Our GRU takes as input a matrix of shape (number of symbols, padded at the beginning of the line) * (size of embedding vector). There are two layers, a layer of GRU neurons and a layer of classification.
A GRU neuron **Figure 6** is a recurrent neuron, i.e. it takes as input not only the current inputs but also its previous output. The problem of such networks is that the information tends to disappear through the epochs: it is the gradient vanishing, the gradient tends to zero and the weights are not updated anymore.

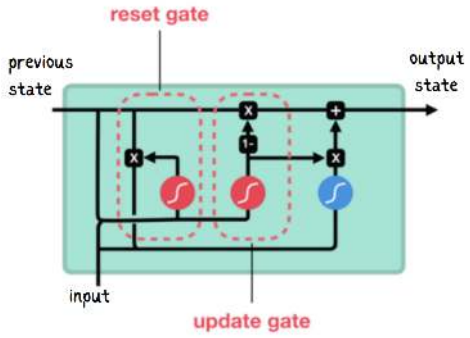


Figure 6: Structure of the GRU neuron

The idea of the GRU neuron to avoid this problem is that, for each neuron, a state matrix is added to keep the information of the previous step. An update and reset gate are fitted to decide what ratio of new information from the next symbol of the line should be used to update the state.
As a final layer, we have a fully connected layer for classification, classifying each sample into one of the 7 dialects.

5 Results

5.0.1 HeLI

HELI was used as baseline of the challenge [11] and achieved a F1-score of 0.71, using a n_{max} of 3 and an unknown value of p.
We also tried to implement the HeLI method but our results were not that great, achieving a f1-score of 0.42 (**Figure 7**) for our best parameters, namely $n_max = 3$ and $p = 0.1$. A difference in preprocessing could explain this difference, or the value of the parameter p. However, we found the best accuracy for the same n_max used in the contest.

Language	LIM	MPG	NsA	NsB	OLB	STB	SUX	TOTAL
F1score, nmax=1	0.4096448	0.3086294	0.057868	0.3598984	0.0020304	0.0025381	0.381218	0.274547
F1score, nmax=2	0.3959398	0.4351774	0.2314721	0.2978746	0.0253807	0.812182	0.2223350	0.369872
F1score, nmax=3	0.7086294	0.7553299	0.3370558	0.2944163	0.1421319	0.4467005	0.2690137	0.42103
F1score, nmax=4	0.485279	0.617259	0.215228	0.2588832	0.254822	0.814721	0.140102	0.326613

Figure 7: f1-scores per dialects for our HeLI

5.0.2 BERT

BERT achieved the best results of the competition with a f1-score of 0.7695. This algorithm is now widely used in a lot of language applications such as language identification, like here, but also word prediction, translation... thanks to its adaptability through transfer learning (reuse already trained layers and train only the last ones) and efficiency, in term of results and computational cost.
In this competition, BERT showed great results after a brief parameter tuning on batch size, number of epochs and learning rate, and the NPCC team found out that a very small number (2) of epochs was mostly sufficient for this algorithm. Others findings were some errors due to the lines with not enough symbols (under 10 for example) and some samples of the development set which were target with several classes (about 1%) [4].

5.0.3 GRU

Our GRU did not achieve such great results, with an overall f1-score of 0.43. Even if we tuned our parameters such as the learning rate **Figure 8**, the results are very low compared to the other teams. Our final parameters were a maximum size of lines of 50, a length of embeddings of 10, a learning rate of 10^3 and a size of hidden layer of 10.

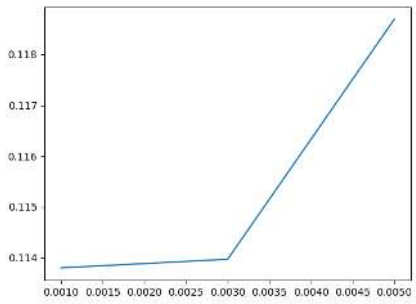
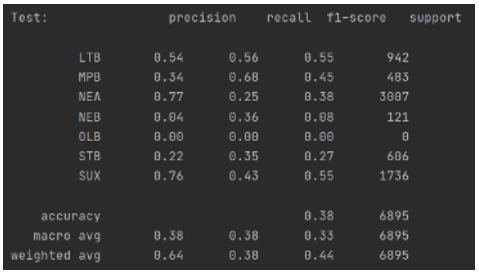


Figure 8: Best Normalized Mean Squared Error among all epochs for each learning rate value

Some explanations for these bad results could be a very unbalanced length of lines which lead us to truncate the size of the lines for computational time reasons, and it could lead to a missclassification of every long-sized lines. Also, like HeLI, the lack of preprocessing could be a problem here. Then, we

did not tune the length of the sign embeddings, which should be done.
However, the total missclassification of a dialect which has good results in other models, Old Babylonian (OLB), maybe points out a code error (**Figure 9**).



Test:	precision	recall	f1-score	support
LTB	0.54	0.56	0.55	942
MPB	0.34	0.68	0.45	483
NEA	0.77	0.25	0.38	3007
NEB	0.04	0.36	0.08	121
OLB	0.00	0.00	0.00	0
STB	0.22	0.35	0.27	606
SUX	0.76	0.43	0.55	1736
accuracy			0.38	6895
macro avg	0.38	0.38	0.33	6895
weighted avg	0.64	0.38	0.44	6895

Figure 9: Scores on each languages for the GRU

6 Conclusion

In conclusion, we have seen several methods of NLP to deal with the CLI dataset, starting from two language representations and two models using them. We also implemented two algorithms which performed badly, but it helped us to understand the issues rose by this dataset about cuneiform writing.
Now, even if this challenge was created notably to improve the state-of-the-art in Natural Language Processing, concerning a kind of languages which were not very explored by algorithms, there are several possibilities to go further. Indeed, if we want to improve the results of the language identification of cuneiform texts, the building of the dataset had some flaws about the handling of the missing or unreadable symbols, and unsupervised methods to impute this data could be used. Then, the use of the images instead of the transliterations of the ORACC dataset could be another possibility, and would be this time a problem of image processing.

References

[1] Alvaro Corrales Cano. Assyrian or babylonian? language identification in cuneiform texts. 2021.

[2] Minoo Nassajian Ehsan Doostmohammadi. Investigating machine learning methods for language and dialect identification of cuneiform texts. Proceedings of the Sixth Workshop on NLP for Similar Languages, Varieties and Dialects:188—-193, 2019.

[3] Cohen et al. Investigating machine learning methods for language and dialect identification of cuneiform texts. *iClay: Digitizing Cuneiform*, The Proceedings of the 5th International Symposium on Virtual Reality, Archaeology and Cultural Heritage (VAST 2004): 135—-143, 2004.

[4] Serge Leger Gabriel Bernier-Colborne, Cyril Goutte. Improving cuneiform language identification with bert. Proceedings of the Sixth Workshop on NLP for Similar Languages, Varieties and Dialects:17–25, 2019.

[5] Marcos Zampieri Gustavo Henrique Paetzold. Experiments in cuneiform language identification. Proceedings of the Third Workshop on NLP for Similar Languages, Varieties and Dialects (VarDial3):153—-162, 2016.

[6] Kenton Lee Jacob Devlin, Ming-Wei Chang and Kristina Toutanova. Bert: pre-training of deep bidirectional transformers for language understanding. CoRR, 2018.

[7] KyungHyun Cho Yoshua Bengio Junyoung Chung, Caglar Gulcehre. Empirical evaluation of gated recurrent neural networks on sequence modeling. NIPS 2014 Deep Learning and Representation Learning Workshop, 2014.

[8] Kwok Him So Pin-zhen Chen Cagrı Coltekin Nianheng Wu, Eric DeMattos. Language discrimination and transfer learning for similar languages: Experiments with feature combinations and adaptation. Proceedings of the Sixth Workshop on NLP for Similar Languages, Varieties and Dialects:54—63, 2019.

[9] Greg Corrado Jeffrey Dean Tomas Mikolov, Kai Chen. Efficient estimation of word representations in vector space. 1st International Conference on Learning Representations, ICLR 2013, 2013.

[10] Heidi Jauhiainen Tommi Jauhiainen, Krister Lindén. Language model adaptation for language and dialect identification of text. *Natural Language Engineering*, 25:561–583, 2019.

[11] Tero Alstola Krister Linden Tommi Jauhiainen, Heidi Jauhiainen. Language and dialect identification of cuneiform texts. Proceedings of the Sixth Workshop on NLP for Similar Languages, Varieties and Dialects:89—-98, 2019.